Shh: Secretly Managing Your SSH Keys

Shh is an elegant command-line toolkit designed for **securely managing SSH keys and secrets** with **AWS Secrets Manager**. It ensures **seamless, automated, and encrypted** storage and retrieval of sensitive credentials, making your DevOps workflow more secure and efficient.

🚀 Features

- 🔐 **Secure SSH Key Storage** – Store and retrieve SSH private keys securely from AWS Secrets Manager.
- ⚡ **Fast & Efficient** – Handles key injection into `ssh-agent` on the fly without writing to disk.
- 🔄 **Seamless Integration** – Works effortlessly with AWS, Ansible, and Terraform.
- 🔍 **Advanced Metadata** – Tracks key details, versions, and automatic rotation schedules.
- 🔁 **Key Rotation** – Monitors key age and suggests rotation timeframes for enhanced security.
- 📎 **Public Key Support** – Upload `.pub` keys alongside private keys for seamless key management.
- 🌍 **Region Flexibility** – Configure AWS regions via CLI arguments or environment variables.
- ⚙️ **Environment Management** – Easily configure, persist, and manage Shh environment variables.
- 🖥️ **Beautiful UI** – Intuitive and visually appealing terminal interface with color-coding.
- 🔧 **Automation Support** – Fully scriptable for CI/CD pipelines and automated deployments.

📦 Installation

Prerequisites

- AWS CLI installed and configured with appropriate permissions
- `jq` for JSON processing (version 1.5+)
- `ssh-agent` running on your system
- `git` for cloning the repository
- Bash shell environment (version 4.0+)

Quick Installation

The easiest way to install Shh is using our installation script:

```
# Basic installation with interactive prompts
curl -fsSL https://raw.githubusercontent.com/jenova-marie/shh/root/shh-install | bash

# Fully automated installation with HTTPS (recommended for CI/CD)
curl -fsSL https://raw.githubusercontent.com/jenova-marie/shh/root/shh-install | bash -s -- --https --auto

# Specify SSH as clone method (if you have GitHub SSH keys configured)
```

```
curl -fsSL https://raw.githubusercontent.com/jenova-marie/shh/root/shh-install | bash -s -- --ssh
```

This will:

1. Download and execute the installer script directly
2. Install all Shh components
3. Create symlinks in `/usr/local/bin`
4. Set proper permissions
5. Log all installation activities to `/var/log/shh.log`
6. Guide you through initial configuration

Installation Options

The installer supports several options to customize the installation process:

Option	Description
`--ssh`	Use SSH for cloning the repository (requires GitHub SSH setup)
`--https`	Use HTTPS for cloning the repository (more reliable for CI/CD)
`--auto`	Fully automated installation with minimal prompts (uses defaults)
`install`	Explicitly specify installation mode (default if not specified)
`--help`	Show usage information and all available options

Examples:

```
# Combine options for customized installation
curl -fsSL https://raw.githubusercontent.com/jenova-marie/shh/root/shh-install | bash -s -- install --https --auto
```

Manual Installation

If you'd like to review the installer before running it (recommended):

```
# Download installation script ONLY (does NOT execute)
curl -fsSL https://raw.githubusercontent.com/jenova-marie/shh/root/shh-install -o shh-install

# Make it executable
chmod +x shh-install

# Run the installer (can add options here too)
./shh-install
```

```
# Example with options
./shh-install --https --auto
```

Uninstallation

To remove Shh from your system:

```
# Interactive uninstallation (with prompts for confirmation)
curl -fsSL https://raw.githubusercontent.com/jenova-marie/shh/root/shh-install | bash -s -- uninstall

# Fully automated uninstallation (no prompts)
curl -fsSL https://raw.githubusercontent.com/jenova-marie/shh/root/shh-install | bash -s -- uninstall --auto

# If you have the script locally
./shh-install uninstall
./shh-install uninstall --auto  # Non-interactive mode
```

The uninstallation process will:

- Remove all symlinks from `/usr/local/bin`
- Delete the installation directory at `/usr/local/share/shh`
- Offer to clean up environment variables from your shell configuration files
- Preserve the log file at `/var/log/shh.log` for reference

⚙️ Configuration

Environment Configuration

You can configure Shh with environment variables:

```
# Regional configuration (in order of precedence)
export SHH_REGION="us-east-2"  # Preferred region
export AWS_REGION="us-east-2"  # Alternative
export AWS_DEFAULT_REGION="us-east-2"  # AWS CLI default

# Secret name configuration
export SHH_SECRETS="my-ssh-keys"  # Name of your AWS Secrets Manager secret

# Debug configuration
export SHH_DEBUG="true"  # Enable debug mode
```

Interactive Configuration (shh-env)

The `shh-env` tool provides a beautiful interactive interface for managing environment variables:

```
# Launch interactive menu
shh-env

# Display current environment configuration
shh-env --display

# Set environment variables for current session
shh-env --set SHH_REGION=us-west-2
shh-env --set SHH_SECRETS=prod-ssh-keys

# Reset all SHH environment variables to defaults
shh-env --reset

# Persist environment variables to your shell config
shh-env --persist

# Enable debug output
shh-env --debug
```

AWS Secret Configuration (shh-admin)

The `shh-admin` tool helps create and manage your AWS Secrets Manager secret:

```
# Launch interactive mode
shh-admin

# Create or verify your AWS secret (non-interactive)
shh-admin --create

# Configure environment variables only
shh-admin --env

# List keys in your secret
shh-admin --list

# All options combined
shh-admin --region us-west-2 --secret prod-keys --list --debug
```

🛠️ Usage

🔑 Securely Add an SSH Key to AWS Secrets Manager

The `shh-add` tool stores your SSH keys in AWS Secrets Manager with rich metadata:

```
# Basic usage - adds key to AWS Secrets Manager
shh-add ~/.ssh/mykey_ed25519 [property-name] [region]

# Add both private and public keys
shh-add ~/.ssh/mykey_ed25519 --pub

# Only upload the specified file, skip public key and fingerprint
shh-add ~/.ssh/mykey_ed25519 --only

# Add key to both AWS Secrets Manager and your local SSH agent
shh-add ~/.ssh/mykey_ed25519 --pub

# Skip adding to SSH agent
shh-add ~/.ssh/mykey_ed25519 --no-ssh-add

# Enable verbose debug output
shh-add ~/.ssh/mykey_ed25519 --debug
```

Key Metadata Features

Each key stored by `shh-add` automatically includes metadata with:

- Key type (ed25519, RSA, etc.) and size (bits)
- Creation and update timestamps
- Version tracking for key rotation history
- SSH key fingerprint for agent identification
- Recommended rotation date (90 days from upload)
- Comments from the original key

🔓 Use SSH Keys with `shh`

The `shh` command retrieves keys from AWS Secrets Manager and uses them with SSH:

```
# Basic syntax (uses username_ed25519 key by default)
shh user@hostname [region] [options]

# Specify a key with -i flag (SSH-style)
shh -i mykey_ed25519 user@hostname [region]

# Standard SSH options are passed through
shh -i mykey_ed25519 -p 2222 user@hostname

# Enable debug output
shh --debug user@hostname

# Configure environment variables
shh --env
```

The `shh` command performs the following steps:

1. Determines which key to use:
    - If specified with `-i`, uses that key name
    - Otherwise, defaults to `username_ed25519` based on the user part of user@host
2. Securely retrieves the key from AWS Secrets Manager
3. Identifies key fingerprint from metadata
4. Checks if the key is already loaded in `ssh-agent`
5. Adds the key to `ssh-agent` in memory (no disk writes) if needed
6. Connects to the specified server

🏗️ Project Architecture

The Shh toolkit consists of several components, each with a specific purpose:

Component	Description
shh	Main command for SSH connections using keys from AWS Secrets Manager
shh-add	Tool for adding SSH keys to AWS Secrets Manager
shh-admin	Administration utility for managing secrets and IAM permissions
shh-env	Environment variable management with beautiful UI
shh-install	Installer/uninstaller script with automation support

Installation Directory Structure

The toolkit is installed in:

- `/usr/local/share/shh/` - Main installation directory containing all scripts
- `/usr/local/bin/` - Symlinks to the scripts for easy command-line access
- `/var/log/shh.log` - System log file for installation and operation events

Design Philosophy

The Shh toolkit follows these design principles:

- **Security First**: No sensitive data written to disk, all operations in memory
- **User Experience**: Beautiful UI with consistent color scheme and formatting
- **Integration**: Works with existing AWS and SSH tools seamlessly
- **Automation**: Full support for CI/CD pipelines and scripted operation
- **Best Practices**: Encourages key rotation and secure credential management

🔄 Key Rotation Best Practices

Shh includes key rotation features:

- Each key automatically has a recommended rotation date (90 days after creation)
- The `shh-admin --list` command shows when each key is due for rotation

- When a key is updated, the rotation history is preserved
- Version numbers are incremented automatically on each update

To rotate a key:

1. Generate a new SSH key: `ssh-keygen -t ed25519 -f ~/.ssh/new_key`
2. Add it to AWS Secrets Manager: `shh-add ~/.ssh/new_key mykey_ed25519 --pub`
3. The previous key data is preserved in the rotation history

🌍 AWS Region Configuration

Region priority (from highest to lowest):

1. Command-line argument (e.g., `shh-add ~/.ssh/mykey_ed25519 mykey us-east-2`)
2. `SHH_REGION` environment variable
3. `AWS_REGION` environment variable
4. `AWS_DEFAULT_REGION` environment variable
5. Default fallback (us-east-2)

🔧 Debug Options

All Shh commands support a `--debug` flag for troubleshooting:

```
shh user@host keyname --debug
shh-add ~/.ssh/mykey --debug
shh-admin --debug
shh-env --debug
```

🛡️ Security Considerations

- No SSH keys are ever written to disk during retrieval
- Keys are securely transmitted from AWS Secrets Manager to SSH agent in memory
- All AWS connections use your authenticated AWS CLI credentials
- Key fingerprints are stored to verify agent-loaded keys without requiring passphrase entry
- All scripts use set -e to ensure they exit immediately on errors

🔍 Troubleshooting

Common Issues

Issue: Script not found after installation
Solution: Check that symlinks were created properly in `/usr/local/bin`

```
ls -la /usr/local/bin/shh*
```

Issue: AWS authentication failures
Solution: Check your AWS credentials and run:

```
aws sts get-caller-identity
```

Issue: SSH agent not running
Solution: Start ssh-agent manually:

```
eval "$(ssh-agent -s)"
```

Logs and Debugging

The main log file is located at:

```
/var/log/shh.log
```

For verbose output, add the `--debug` flag to any command:

```
shh --debug user@host
```

🌍 Open Source & Contributions

We welcome contributions, improvements, and suggestions for enhancements.

```
git clone git@github.com:jenova-marie/shh.git
```

Pull requests and issues are welcome!

📝 License

Shh is released under the **MIT License**.

AWS IAM Permissions Required

The following AWS IAM permissions are required for Shh to function properly:

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
```

```
            "Effect": "Allow",
            "Action": [
                "secretsmanager:CreateSecret",
                "secretsmanager:GetSecretValue",
                "secretsmanager:PutSecretValue",
                "secretsmanager:UpdateSecret",
                "secretsmanager:DescribeSecret",
                "secretsmanager:ListSecrets"
            ],
            "Resource": "arn:aws:secretsmanager:*:*:secret:YOUR-SECRET-
NAME-*"
        }
    ]
}
```

Replace `YOUR-SECRET-NAME` with your actual secret name (e.g., `ssh-keys`). For secrets with path-like structures (e.g., `Test/X/123`), use the full path in the resource name.

You can attach this policy to your IAM user or role through the AWS Management Console or AWS CLI.